MoMoGuard

71 questions que le jury peut poser — et les réponses

Préparation à la soutenance — Projet de fin d'études, Licence 4
Détection de fraude Mobile Money : phishing, smishing, ingénierie sociale

1.	Généralités et contexte	8 questions
2.	Architecture et conception	8 questions
3.	Moteur de scoring et détection	10 questions
4.	Application Android	8 questions
5.	Serveur, dashboard et base de données	8 questions
6.	Sécurité de la plateforme	7 questions
7.	Éthique, légalité et consentement	6 questions
8.	Évaluation, métriques et tests	8 questions
9.	Limites, perspectives et questions pièges	8 questions

Conseil général : ne récite pas. Lis chaque réponse, reformule-la avec tes mots, et entraîne-toi à voix haute. Les encadrés en italique signalent les questions les plus probables.

1. Généralités et contexte

Q1. Présentez votre projet en une phrase.

MoMoGuard est une plateforme consentie de détection de fraude Mobile Money : des agents installés sur les propres appareils de l'utilisateur analysent SMS, notifications et liens suspects, les notent avec un moteur de scoring explicable, et remontent les alertes en temps réel à un serveur de supervision.

» Conseil : Entraîne-toi à la dire en moins de 20 secondes, sans regarder tes notes.

Q2. Pourquoi avoir choisi ce sujet ?

En RDC, le Mobile Money (Orange Money, Airtel Money, M-Pesa, Africell) est le principal moyen de paiement de millions de personnes. Les fraudeurs n'attaquent pas la technologie des opérateurs mais l'utilisateur lui-même, par smishing et ingénierie sociale, et il n'existe presque aucun outil de protection adapté au contexte local.

Q3. Quel est l'axe précis de votre travail ?

La sécurité des utilisateurs humains — phishing, smishing, ingénierie sociale — et non la sécurité des infrastructures des opérateurs. C'est un choix assumé : le maillon faible exploité dans la fraude Mobile Money, c'est la personne.

Q4. Qu'est-ce que le smishing exactement ?

Le phishing par SMS : un message texte qui usurpe une identité de confiance (opérateur, banque, proche) pour pousser la victime à cliquer sur un lien, communiquer un code ou effectuer un transfert.

Q5. À qui s'adresse votre solution ?

À deux profils : l'utilisateur final de Mobile Money, protégé par l'application Android sur son téléphone, et le superviseur ou analyste, qui suit les alertes du parc via le tableau de bord central.

Q6. Pourquoi insistez-vous sur le mot « consentie » ?

Parce qu'un système qui lit des SMS peut vite devenir un outil de surveillance. MoMoGuard n'est installé que sur les propres appareils de l'utilisateur, après consentement explicite, enregistré et révocable à tout moment ; le serveur refuse tout rapport venant d'un appareil non consenti.

» Conseil : C'est probablement LA question éthique du jury : réponds sans te justifier, c'est une force du projet.

Q7. En quoi est-ce différent d'un simple filtre antispam ?

Trois différences : l'adaptation au contexte congolais (opérateurs locaux, vocabulaire, codes USSD, numéros +243), l'explicabilité (chaque alerte est justifiée règle par règle), et la chaîne complète : capture sur l'appareil, scoring central, supervision temps réel et notification push de retour.

Q8. Quelles technologies principales avez-vous utilisées ?

Kotlin pour l'application Android ; Python/Flask avec SocketIO pour le serveur et le dashboard ; SQLAlchemy pour la persistance (SQLite, PostgreSQL ou MySQL) ; Firebase pour l'authentification mobile et les notifications push FCM ; pytest pour les 58 tests automatisés.

2. Architecture et conception

Q9. Décrivez l'architecture générale du système.

Deux agents (Android en Kotlin, Windows en Python) envoient les évènements suspects au serveur Flask via POST /report. Le serveur les analyse avec le moteur de scoring, les stocke, pousse l'alerte au dashboard en WebSocket et renvoie une notification push FCM au téléphone de l'utilisateur.

Q10. Pourquoi un serveur central plutôt que tout faire sur le téléphone ?

Le téléphone fait déjà un pré-filtrage local, mais le serveur apporte ce qu'un appareil isolé ne peut pas offrir : supervision d'un parc entier, statistiques agrégées, mise à jour des règles et des seuils sans redéployer l'application, et corrélation entre utilisateurs.

Q11. Pourquoi Flask et pas Django ou FastAPI ?

Flask est léger et parfaitement adapté à un prototype de cette taille : je maîtrise chaque route et chaque middleware. Django aurait imposé beaucoup de structure inutile ; FastAPI était une alternative crédible, mais Flask-SocketIO offrait le temps réel dont j'avais besoin avec une intégration simple.

Q12. Comment le temps réel est-il assuré sur le dashboard ?

Par WebSocket avec Flask-SocketIO : à chaque nouvelle alerte, le serveur émet un évènement que le navigateur affiche instantanément, sans rechargement de page. C'est ce que je montre pendant la démonstration.

Q13. Pourquoi SQLAlchemy et trois SGBD supportés ?

SQLAlchemy rend la base portable : SQLite en développement (zéro configuration), PostgreSQL ou MySQL en production, sélectionnés par une simple variable d'environnement MOMOGUARD_DB. Les tables sont créées au démarrage et les colonnes ajoutées au fil des versions sont migrées sans perte de données.

Q14. Que contient exactement un rapport envoyé par un agent ?

Le type d'évènement (sms, notification, clipboard, file), le contenu textuel, des métadonnées comme l'expéditeur et la plateforme, et l'identité de l'agent. Le serveur y ajoute le score, le niveau et la liste des raisons après analyse.

Q15. Comment les agents s'authentifient-ils auprès du serveur ?

L'agent doit être enregistré et approuvé côté serveur ; côté mobile, l'utilisateur s'authentifie via Firebase et le serveur vérifie le jeton d'identité à chaque appel. L'endpoint /report contrôle aussi l'adhésion : un agent bloqué ou un utilisateur non consenti est refusé.

Q16. Que fait l'agent Windows ?

Il surveille deux vecteurs propres au PC : le presse-papiers, pour détecter la substitution d'un numéro de compte copié (attaque du bénéficiaire modifié), et les téléchargements, pour signaler les fichiers exécutables dangereux (.exe, .apk, .bat...).

3. Moteur de scoring et détection

Q17. Comment fonctionne votre moteur de détection ?

C'est un moteur à base de règles, additif et explicable : 14 règles inspectent le message ; chaque signal suspect ajoute des points ; le score est borné entre 0 et 100 ; deux seuils configurables (25 et 55) donnent le niveau faible, moyen ou élevé ; et chaque règle déclenchée est enregistrée pour justifier la décision.

Q18. Pourquoi des règles plutôt que du machine learning ?

Quatre raisons : l'explicabilité (chaque alerte est justifiable devant l'utilisateur et devant vous), l'absence de grand dataset étiqueté de SMS frauduleux congolais, la maîtrise complète du comportement, et la rapidité d'exécution. Le ML est une perspective d'hybridation, pas un point de départ réaliste pour ce contexte.

» Conseil : Question quasi certaine. Ne sois pas défensif : c'est un choix d'ingénierie argumenté.

Q19. Citez quelques règles et leurs poids.

Lien vers une adresse IP brute : +30 ; lien raccourci : +20 ; extension de domaine douteuse (.xyz, .tk...) : +15 ; domaine sosie d'un opérateur : +35 ; vocabulaire d'arnaque : jusqu'à +32 ; urgence combinée à un lien : +12 ; code USSD : +25 ; référence à un SIM swap : +25 ; demande de partage d'un code : +30 ; faux agent opérateur : +25 ; arnaque au remboursement : +25 ; appât au gain : +25 ; promesse de multiplication d'argent : +22 ; frais de déblocage : +22.

» Conseil : Inutile de tout réciter : 4 ou 5 exemples variés suffisent.

Q20. Qu'est-ce qu'un homoglyphe et comment le détectez-vous ?

Un caractère qui imite visuellement une lettre : « 0range » avec un zéro, « airte1 » avec un un. Le moteur translittère le domaine (0 devient o, 1 devient i, 3 devient e...), le compare aux noms d'opérateurs, et vérifie qu'il ne s'agit pas d'un domaine officiel : « 0range-money.xyz » est lu « orangemoney » et déclenche la règle du domaine sosie.

Q21. Un message légitime de prévention comme « ne partagez jamais votre code » ne déclenche-t-il pas une alerte ?

Non : la règle de demande de code est neutralisée par une liste de formulations protectrices (« jamais », « à personne », « ne communiquez pas »...). C'est précisément ce qui explique le zéro faux positif de l'évaluation : les vrais messages des opérateurs contiennent ces mises en garde.

Q22. Comment avez-vous choisi les poids des règles ?

Empiriquement, selon deux critères : la gravité du signal (un domaine sosie est presque toujours frauduleux : +35) et sa spécificité (le http seul est faible : +5). Les poids ont ensuite été ajustés en itérant sur le jeu d'évaluation pour maximiser le compromis précision/rappel.

Q23. Pourquoi les seuils 25 et 55 ?

Ils résultent du benchmark : 25 capte les combinaisons de signaux faibles sans déclencher sur un signal isolé anodin, 55 réserve le niveau « élevé » aux messages multi-signaux quasi certains. Ils sont configurables depuis le dashboard, donc ajustables sans toucher au code.

Q24. Le moteur résiste-t-il aux variations d'écriture (accents, majuscules) ?

Oui : le texte est normalisé en Unicode NFD, les accents sont supprimés et tout passe en minuscules. « GAGNÉ », « gagne » et « Gagné » sont traités identiquement.

Q25. Et si le message est en lingala ou en swahili ?

C'est une limite actuelle : le lexique est français. Les règles structurelles restent efficaces (URL, homoglyphes, USSD, montants, numéros), mais les règles lexicales ne s'appliquent pas. L'ajout de lexiques multilingues est une perspective directe, facilitée par l'architecture du moteur.

Q26. Qu'apporte l'intégration de Google Safe Browsing ?

Une vérification de réputation des URL contre la base mondiale de Google : une URL confirmée malveillante classe l'alerte « élevée » d'office. Elle est optionnelle et dégradable : sans clé API ou hors ligne, la détection par règles continue normalement — le système ne dépend d'aucun service externe.

4. Application Android

Q27. Comment l'application capture-t-elle les SMS ?

Par un BroadcastReceiver enregistré sur l'évènement SMS_RECEIVED avec la permission RECEIVE_SMS. À la réception, le message passe d'abord par le filtre de pertinence local avant toute transmission.

Q28. Comment captez-vous les messages WhatsApp ?

Par un NotificationListenerService : l'application lit le contenu des notifications affichées — pas l'intérieur des applications — après une autorisation système explicite que l'utilisateur accorde manuellement dans les paramètres Android.

Q29. L'application envoie-t-elle tous les messages au serveur ?

Non. Un filtre de pertinence local (classe FiltreMessagesFraude) ne retient que les messages liés au Mobile Money — confirmations d'opérateur ou suspicion de smishing — et écarte immédiatement les messages personnels, sans trace. C'est le principe de minimisation des données.

» Conseil : Point fort à placer dès que le jury s'inquiète de la vie privée.

Q30. Pourquoi un service au premier plan (foreground service) ?

Pour garantir la surveillance continue malgré les optimisations de batterie d'Android, qui tuent les services d'arrière-plan. La notification permanente qu'il impose est aussi une garantie de transparence : l'utilisateur voit que la protection est active.

Q31. Comment l'utilisateur s'authentifie-t-il ?

Compte Firebase (email/mot de passe) renforcé par une MFA par SMS, puis validation manuelle du compte par l'administrateur avant activation. Trois barrières avant qu'un appareil ne rejoigne le parc.

Q32. Comment les données locales sont-elles protégées ?

Les préférences (jeton, URL du serveur) sont stockées dans EncryptedSharedPreferences via androidx.security.crypto, et allowBackup est désactivé pour empêcher l'exfiltration par sauvegarde système.

Q33. Comment le téléphone reçoit-il les alertes ?

L'application enregistre son jeton FCM auprès du serveur ; quand une fraude est confirmée, le serveur envoie une notification push via Firebase Cloud Messaging, reçue par le service de messagerie de l'application même si elle est fermée.

Q34. L'application communique-t-elle en clair avec le serveur ?

Non : le code exige https, avec une seule exception encadrée — le http n'est accepté qu'en build de debug, pour le développement local. En production, tout transite chiffré en TLS.

5. Serveur, dashboard et base de données

Q35. Que voit-on sur le tableau de bord ?

Les alertes en direct avec score, niveau et raisons ; des statistiques (répartition par niveau, par type, par plateforme) ; et l'accès aux pages de gestion : utilisateurs, agents, consentements, audit, paramètres des seuils.

Q36. À quoi sert la page Analyzer ?

À analyser manuellement n'importe quel message : on colle un texte, le moteur le score en direct et affiche les raisons. C'est utile à l'analyste, et c'est aussi mon plan B de démonstration si le réseau mobile fait défaut.

Q37. Comment gérez-vous les utilisateurs ?

Chaque compte Firebase apparaît dans la page Utilisateurs : l'admin approuve, rejette ou bloque, consulte la fiche détaillée, modifie les informations et peut réinitialiser un mot de passe — l'utilisateur en est alors notifié.

Q38. Que trace le journal d'audit ?

Chaque action administrateur : connexions, approbations, blocages, changements de paramètres, révocations de consentement... avec horodatage. Aucune action sensible n'est anonyme.

Q39. Pourquoi SQLite en développement et PostgreSQL en production ?

SQLite est parfait pour développer et démontrer : un fichier, zéro installation. PostgreSQL apporte en production la concurrence d'accès, la robustesse et la montée en charge. SQLAlchemy rend le passage transparent : c'est une variable d'environnement.

Q40. Comment sauvegardez-vous les données ?

Un script dédié exporte toutes les tables en JSON horodaté. C'est une sauvegarde logique, indépendante du SGBD, restaurable et versionnable.

Q41. Que fait le bouton « Simuler une alerte » ?

Il insère une alerte réaliste via la route /demo et la pousse par le circuit normal — WebSocket et FCM. Il simule uniquement l'envoi du SMS ; tout le reste de la chaîne (scoring, stockage, temps réel, push) est réel. C'est mon filet de sécurité de démonstration, et je l'annonce honnêtement si je l'utilise.

Q42. Peut-on exporter les données pour analyse externe ?

Oui, les alertes s'exportent en CSV depuis le dashboard, pour exploitation dans un tableur ou un outil d'analyse.

6. Sécurité de la plateforme

Q43. Comment le compte administrateur est-il protégé ?

Mot de passe hashé, double authentification TOTP (configurée par QR code, compatible Google Authenticator), verrouillage temporaire après échecs répétés (anti force brute), et journal d'audit de chaque action.

Q44. Comment empêchez-vous l'injection de faux rapports dans /report ?

Trois contrôles : l'agent doit être enregistré et non bloqué, le jeton Firebase de l'utilisateur est vérifié côté serveur, et l'adhésion/consentement est contrôlée. Un rapport qui échoue à l'un des trois est rejeté.

Q45. Êtes-vous protégé contre l'injection SQL ?

Oui : toutes les requêtes passent par l'ORM SQLAlchemy, qui paramètre systématiquement les valeurs. Aucune requête n'est construite par concaténation de chaînes.

Q46. Et contre le XSS sur le dashboard ?

Les templates Jinja2 échappent automatiquement le contenu affiché — important puisque le dashboard affiche des messages écrits par des fraudeurs, donc potentiellement hostiles. C'est couvert par les tests de sécurité.

Q47. Comment gérez-vous les secrets (clés API, Firebase) ?

La clé de service Firebase (serviceAccountKey.json) et la clé Safe Browsing ne sont jamais dans le code ni dans le dépôt git : fichier exclu par .gitignore et variables d'environnement. Le dépôt est publiable sans fuite.

Q48. Que se passerait-il si votre serveur était compromis ?

L'impact est limité par conception : données minimisées (pas de messages personnels), mots de passe hashés, jetons révocables, sauvegardes restaurables. Je reconnais une amélioration possible : le chiffrement au repos de la base, prévu en perspective.

» Conseil : Admettre une limite avec son plan de correction vaut mieux que prétendre l'invulnérabilité.

Q49. Votre application pourrait-elle être attaquée elle-même ?

Oui, tout logiciel de sécurité est une cible. C'est pourquoi : https obligatoire hors debug, préférences chiffrées, MFA, validation admin des comptes, et minimisation des privilèges (l'app ne lit pas les SMS envoyés, uniquement les reçus).

7. Éthique, légalité et consentement

Q50. Lire les SMS des gens, n'est-ce pas de la surveillance ?

Non, et la nuance est structurelle : l'utilisateur installe lui-même l'agent sur son propre appareil, donne un consentement explicite et révocable, et le filtre local écarte les messages personnels sans trace. C'est un antivirus comportemental, pas un outil d'écoute : personne d'autre que l'utilisateur ne décide de la protection de son appareil.

» Conseil : Question presque garantie. Réponds calmement, en trois points : consentement, propre appareil, minimisation.

Q51. Comment fonctionne concrètement le consentement ?

Un formulaire public (/consent) enregistre le consentement en base ; l'admin peut consulter la liste et l'utilisateur peut révoquer à tout moment. L'endpoint /report vérifie l'adhésion : sans consentement valide, aucun rapport n'est accepté.

Q52. Que se passe-t-il si l'utilisateur révoque son consentement ?

La révocation est immédiate : l'agent est refusé sur /report et plus aucune donnée ne remonte. L'utilisateur peut aussi simplement désactiver la protection ou désinstaller l'application.

Q53. Votre projet respecte-t-il la réglementation sur les données personnelles ?

Il applique les principes du RGPD, référence internationale, et s'inscrit dans l'esprit du Code du numérique congolais (2023) : consentement préalable, minimisation des données, droit de retrait, traçabilité des traitements. Une mise en conformité formelle ferait partie d'un déploiement réel.

Q54. Stockez-vous les messages légitimes des utilisateurs ?

Les messages personnels sont écartés sur le téléphone, sans transmission ni trace. Seuls les évènements pertinents (liés au Mobile Money) remontent, et le contenu d'une alerte est conservé pour pouvoir justifier la décision — c'est la contrepartie de l'explicabilité.

Q55. Pourquoi l'administrateur doit-il valider chaque compte ?

Pour contrôler le parc : éviter les inscriptions frauduleuses ou les agents pirates, et garder une traçabilité de qui est protégé. C'est cohérent avec un déploiement pilote supervisé.

8. Évaluation, métriques et tests

Q56. Comment avez-vous évalué votre moteur ?

Avec un jeu de 48 messages étiquetés (26 frauduleux, 22 légitimes), représentatifs du contexte Mobile Money congolais. Un message est considéré détecté à partir du niveau « moyen ». Le script d'évaluation est versionné : le benchmark est entièrement reproductible.

Q57. Rappelez vos résultats chiffrés.

Rappel (taux de détection) : 96,2 % ; précision : 100 % ; faux positifs : 0 % ; F1-score : 98,0 % ; exactitude : 97,9 %. Soit 25 fraudes détectées sur 26, et aucun message légitime signalé à tort.

» Conseil : Connais ces cinq chiffres par cœur.

Q58. Expliquez la différence entre précision et rappel.

Le rappel mesure la part des fraudes réelles que je détecte (25/26 = 96,2 %) ; la précision mesure la part de mes alertes qui sont de vraies fraudes (25/25 = 100 %). Un rappel faible laisse passer des fraudes ; une précision faible noie l'utilisateur sous les fausses alertes et détruit sa confiance.

Q59. Pourquoi privilégier zéro faux positif plutôt qu'un rappel parfait ?

Parce qu'un système qui crie au loup est désinstallé en une semaine. La confiance de l'utilisateur est l'actif principal d'un outil de protection grand public : mieux vaut manquer marginalement une fraude que de signaler à tort les vrais messages de son opérateur.

Q60. Parlez-nous de votre faux négatif.

« Promo spéciale : envoyez 10$ et recevez 500$ sur votre compte M-Pesa aujourd'hui » — pas de lien, pas de demande de code, pas d'urgence : il passait sous les seuils. L'analyse de cette erreur a produit la règle « promesse de multiplication d'argent » (+22). C'est le cycle vertueux : évaluer, comprendre l'erreur, corriger.

» Conseil : Transforme la faiblesse en démonstration de méthode.

Q61. 48 messages, n'est-ce pas trop peu pour conclure ?

Si, et je l'assume : les vrais SMS frauduleux sont difficiles à collecter légalement et éthiquement. Les métriques sont donc indicatives, pas définitives. Le protocole, lui, est solide et reproductible ; étendre le dataset par une collecte de terrain est la première de mes perspectives.

Q62. Que couvrent vos 58 tests automatisés ?

Le moteur de scoring, la 2FA, le verrouillage anti force brute, le contrôle d'adhésion sur /report, la gestion des utilisateurs et des agents, les consentements, la threat intelligence, les pages du dashboard, le filtre de l'agent Windows... Ils tournent avec pytest en une commande.

Q63. Comment évalueriez-vous le système en production ?

Par les signaux de terrain : taux de faux positifs signalés par les utilisateurs, taux de confirmation des URL par Safe Browsing, évolution de la distribution des scores, et taux de rétention de l'application — le meilleur indicateur de confiance.

9. Limites, perspectives et questions pièges

Q64. Quelles sont les principales limites de votre travail ?

Trois, assumées : le jeu d'évaluation réduit (48 messages), des règles statiques face à des fraudes qui évoluent, et la couverture limitée à Android et Windows. Chacune ouvre directement une perspective : collecte de terrain, hybridation avec du ML, agent iOS.

» Conseil : Annonce-les toi-même avant qu'on te les oppose : c'est toi qui contrôles le récit.

Q65. Un fraudeur qui connaît vos règles peut-il les contourner ?

Partiellement : il peut changer de vocabulaire. Mais les règles structurelles sont difficiles à éviter sans casser l'arnaque elle-même : une arnaque a besoin d'un lien, d'un code demandé, d'un transfert ou d'une pression — exactement ce que je détecte. Et les règles se mettent à jour côté serveur, sans redéployer l'application. À terme, l'hybridation avec du ML réduit encore cette surface.

Q66. Pourquoi ne pas avoir utilisé du deep learning, c'est l'état de l'art ?

L'état de l'art académique, oui — mais pas pour ce problème dans ce contexte : pas de dataset congolais de dizaines de milliers de SMS étiquetés, besoin d'explicabilité devant des utilisateurs non techniciens, et coût d'infrastructure. Un modèle appris viendra compléter les règles quand la plateforme aura collecté assez de données — la plateforme elle-même est l'outil de collecte.

Q67. Votre système passe-t-il à l'échelle ?

Pour un pilote (centaines d'agents), oui : Flask, SocketIO et PostgreSQL suffisent largement. Au-delà, l'architecture est déjà découplée : il faudrait des workers multiples derrière un load balancer et une file de messages pour absorber les pics de /report. Rien dans la conception ne s'y oppose.

Q68. En quoi votre solution diffère-t-elle de ce que font déjà les opérateurs ?

Les opérateurs font de l'anti-fraude interne, sur leur propre réseau, et ne voient pas ce qui arrive par SMS ordinaire ou WhatsApp. MoMoGuard protège l'utilisateur côté terminal, tous opérateurs confondus — les deux approches sont complémentaires, d'où la perspective de partenariat.

Q69. Si vous aviez six mois de plus, que feriez-vous ?

Dans l'ordre : une campagne de collecte de SMS frauduleux réels avec des utilisateurs volontaires ; l'hybridation règles + classifieur ML entraîné sur ces données ; des lexiques lingala et swahili ; un agent iOS ; et le chiffrement au repos de la base.

Q70. Ce projet est-il déployable en l'état ?

En pilote supervisé, oui : la chaîne complète fonctionne, elle est testée et documentée. Pour un déploiement grand public, il faudrait le passage en PostgreSQL avec TLS de bout en bout, une distribution signée de l'APK, la mise en conformité réglementaire formelle et le support multilingue.

Q71. Qu'avez-vous appris de plus important dans ce projet ?

Trois choses : techniquement, la construction d'une chaîne complète — mobile, serveur, temps réel, push ; méthodologiquement, la valeur d'un benchmark reproductible et de tests automatisés qui m'ont plusieurs fois évité des régressions ; et humainement, qu'un système de sécurité ne vaut que si les utilisateurs lui font confiance — d'où l'explicabilité et le consentement au centre de la conception.

» Conseil : Termine par cette réponse si on te la pose en dernier : elle conclut parfaitement.